

DOCUMENTO DE ARQUITETURA E ESPECIFICAÇÃO TÉCNICA

1. Objetivo do Documento

Definir a arquitetura e as decisões de engenharia para a aplicação web de gestão de documentos, estabelecendo diretrizes claras para o desenvolvimento, execução local e deploy em ambiente público, conforme exigido em edital.

2. Decisões Arquiteturais e Justificativas

2.1. Backend: Python com FastAPI

A aplicação utilizará Python em conjunto com o framework FastAPI.

Justificativa: A escolha atende à permissão de linguagem livre. O FastAPI foi selecionado por fornecer tipagem estrita e validação de dados de entrada via Pydantic. A geração automática da especificação OpenAPI (Swagger) cumpre os requisitos de organização e legibilidade, facilitando a verificação das rotas pela equipe avaliadora. O framework gerencia requisições assíncronas e o processamento de arquivos binários de forma nativa e eficiente.

2.2. Banco de Dados: SQLite

O armazenamento de dados estruturados será realizado via SQLite.

Justificativa: Atende à exigência de estrutura mínima para relacionamentos. Sendo um banco de dados relacional contido em um único arquivo, elimina a necessidade de configuração de servidores de banco de dados no ambiente local do avaliador. Isso garante que as instruções de execução local sejam objetivas e livres de erros de dependência externa.

2.3. Frontend: HTML5, CSS3 e JavaScript

A interface de usuário será construída exclusivamente com HTML, CSS e JavaScript nativos.

Justificativa: Cumpre estritamente o requisito de não exigir layout avançado. A ausência de bibliotecas de terceiros ou frameworks de interface previne complexidade desnecessária para o escopo proposto. A integração com o backend será feita nativamente pela Fetch API, demonstrando capacidade de manipular o DOM e gerenciar estados de forma fundamental.

2.4. Armazenamento de Arquivos: Sistema de Arquivos (File System) Local

Os documentos enviados (PDF, JPG, PNG) serão salvos no diretório interno do servidor.

Justificativa: O edital determina o armazenamento local dos arquivos. O ambiente de deploy será configurado em uma infraestrutura que permita persistência de disco, garantindo que os arquivos físicos não sejam excluídos após períodos de inatividade do servidor, mantendo a integridade com os registros do banco de dados.

3. Modelagem de Dados

O esquema relacional é composto por duas entidades para garantir a integridade entre os metadados do arquivo e seus respectivos comentários.

Entidade: documentos

- id: Integer (Chave Primária, Autoincremento)
- titulo: String (Obrigatório, não nulo)
- descricao: String (Opcional, aceita nulo)
- nome_arquivo_original: String (Obrigatório, não nulo)
- caminho_armazenamento: String (Obrigatório, não nulo)
- data_upload: DateTime (Obrigatório, gerado automaticamente)

Entidade: comentarios

- id: Integer (Chave Primária, Autoincremento)
- documento_id: Integer (Chave Estrangeira referenciando documentos.id, Obrigatório)
- texto: Text (Obrigatório, não nulo)

- `data_hora_registro`: DateTime (Obrigatório, gerado automaticamente)

4. Especificação dos Endpoints (API)

As rotas a seguir expõem as operações necessárias para o funcionamento do sistema.

- **POST `/api/documentos/`**
Recebe um formulário multipart (multipart/form-data) contendo o arquivo físico, o título e a descrição. Valida a extensão do arquivo, grava no disco local e insere os metadados no SQLite.
- **GET `/api/documentos/`**
Consulta o banco de dados e retorna uma lista em formato JSON contendo o id, título e data de upload de todos os documentos cadastrados.
- **GET `/api/documentos/{id}/download`**
Localiza o registro pelo id, busca o arquivo correspondente no sistema de arquivos local e retorna o arquivo binário para visualização ou download pelo navegador.
- **POST `/api/documentos/{id}/comentarios`**
Recebe um payload JSON contendo o texto do comentário. Valida a existência do documento correspondente e insere o registro no banco de dados.
- **GET `/api/documentos/{id}/comentarios`**
Retorna uma lista em formato JSON com todos os comentários vinculados a um documento específico, ordenados cronologicamente.

5. Estrutura de Diretórios

A base de código segue o princípio de separação de responsabilidades.

```
/
├── backend/
│   ├── main.py
│   ├── models.py
│   ├── database.py
│   └── uploads/
├── frontend/
└── index.html
```

```
| |— documento.html
| |— css/
| |   |— style.css
| |   |— js/
| |       |— app.js
|— requirements.txt
|— .gitignore
|— README.md
```

6. Roteiro de Implementação

A execução seguirá ordem sequencial para garantir validação contínua.

1. ****Configuração de Ambiente:**** Criação do repositório Git, inicialização do ambiente virtual Python e definição do .gitignore.
2. ****Camada de Dados:**** Definição dos modelos relacionais e rotinas de conexão com o banco SQLite.
3. ****Camada de Aplicação (Backend):**** Implementação e teste dos endpoints FastAPI via interface Swagger nativa.
4. ****Camada de Apresentação (Frontend):**** Estruturação do HTML e aplicação de regras CSS para garantir organização visual.
5. ****Integração:**** Desenvolvimento da comunicação assíncrona entre o JavaScript do frontend e a API do backend.
6. ****Deploy e Documentação:**** Configuração do ambiente de produção com persistência de disco, implantação da aplicação e elaboração do README.md com instruções de execução.